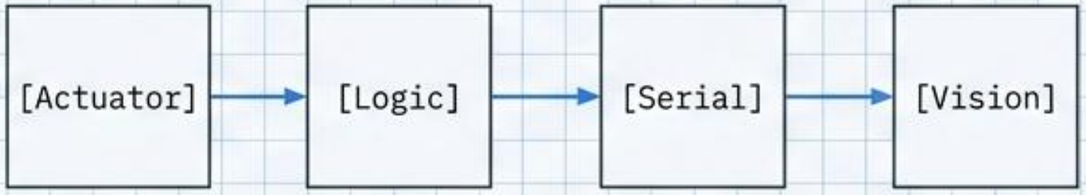
Vision-Based Hardware Control

ROS2 Systems Integration, Node Isolation, and ESP32 Serial Bridging (PW3)




```
Environment:    Ubuntu 24.04 / ROS2 Jazzy
Workspace:      ~/mindspark_ws/src/mindspark_robot
ROS_DOMAIN_ID: 30
```

Gemini Notebook

The ROS2 Macro Architecture

Four isolated nodes. One continuous data pipeline.

Stage 1

SENSOR

Hardware:
USB Webcam

Stage 2

NODES

Publishes:
/camera/image_raw
(sensor_msgs/Image)

Publishes:
/green_ball
(std_msgs/String)

Publishes:
/vision/decision
(std_msgs/String)

1

camera_node

2

green_ball_node

3

vision_decision_node

4

esp32_control_node

Stage 3

USB Serial (/dev/ttyUSB0)

Stage 4

ACTUATOR

Gemini Notebook

The Hardware-Software Handshake Matrix

Camera Input (X-Position)	ROS2 Topic Msg (/vision/decision)	Serial Command (Char)	ESP32 GPIO Pin	Physical Output (State)
x = 0–105	Data: 'L'	'L' (115200 baud)	GPIO 25	RED LED ON
x = 106–212	Data: 'C'	'C' (115200 baud)	GPIO 26	YELLOW LED ON
x = 213–319	Data: 'R'	'R' (115200 baud)	GPIO 27	GREEN LED ON
No detection / Timeout	Data: 'N'	'N' (115200 baud)	N/A	ALL LEDs OFF

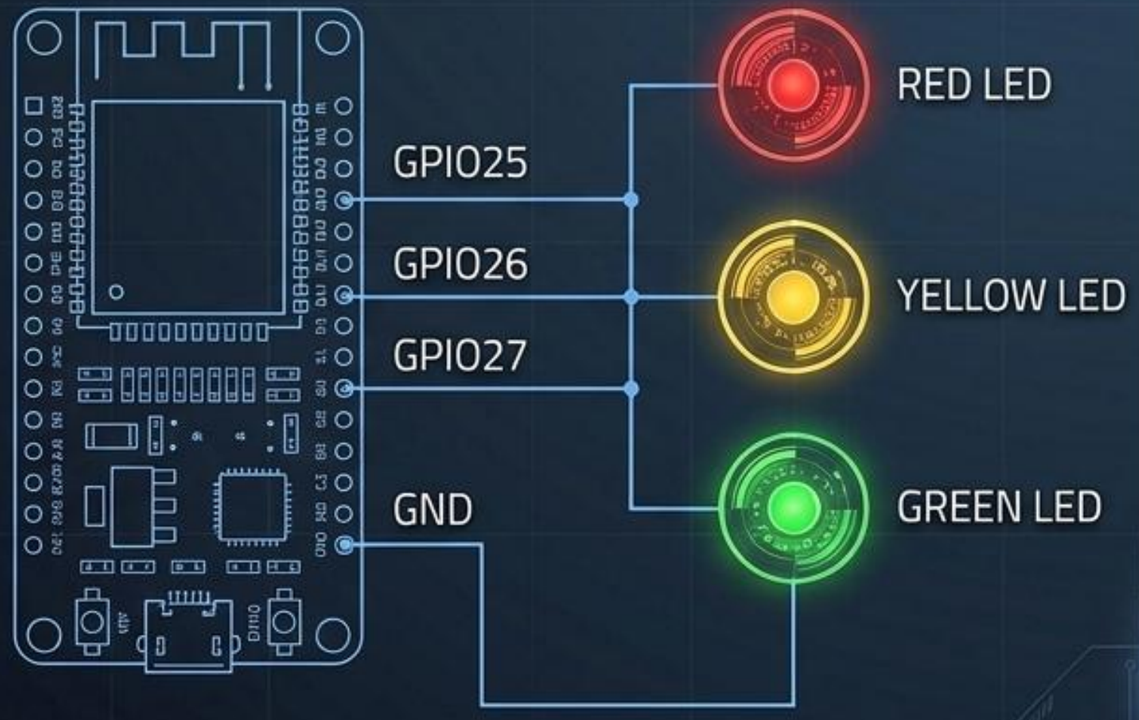
Diagnostic Baseline: Camera resolution 320×240.
Decision publishing target: 5 Hz.

Gemini Notebook

The Physical Handover

Test via Arduino Serial Monitor (115200 baud) first. Port must be closed before ROS2 assumes control.

Hardware: ESP32 Schematic



Firmware: Serial Listener

ESP32 Serial Listener (Arduino IDE)

```
1  char command = Serial.read();
2  allLEDsOff();
3
4  switch (command) {
5    case 'L': case 'l': digitalWrite(25, HIGH); break;
6    case 'C': case 'c': digitalWrite(26, HIGH); break;
7    case 'R': case 'r': digitalWrite(27, HIGH); break;
8    case 'N': case 'n': break; // allLEDsOff() remains active
9  }
```

Gemini Notebook

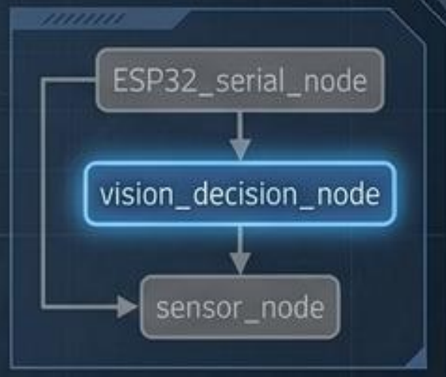
Spatial Mapping Transformation



frame_width = 320
left_boundary = 320 // 3 = 106
right_boundary = (320 * 2) // 3 = 213

Gemini Notebook

Internal Logic: vision_decision_node.py




```python
def ball_callback(self, msg):
    data = msg.data.strip()

    # Regex extraction of X coordinate
    match = re.search(r'\bx\s*=\s*(\d+)', data)
    x = int(match.group(1))

    # Spatial boundary evaluation
    if x < self.left_boundary: self.current_decision = 'L'
    elif x < self.right_boundary: self.current_decision = 'C'
    else: self.current_decision = 'R'
```

Translates standard string 'DETECTED: x=150, y=120' into actionable integers.

Gemini Notebook

Inspecting the Active ROS2 Graph

Check Nodes

```
user@ros2-host:~$ ros2 node list
/ESP32_serial_node
/sensor_node
/vision_decision_node
/rviz2
/robot_state_publisher
```

Echo Output

```
user@ros2-host:~$ ros2 topic echo /vision/decision
---
data: 'L'
---
data: 'L'
---
data: 'C'
---
data: 'R'
---
data: 'R'
---
data: 'N'
---
data: 'N'
```

- Frequency Target: ros2 topic hz /vision/decision should read ~5 Hz.
- Domain Strictness: Rule: export ROS_DOMAIN_ID=30 must be uniform across all terminals to ensure topic visibility.

Gemini Notebook

The Serial Bridge Concept

Crossing from Linux into embedded hardware.

Linux / ROS2 Workspace

esp32_control_node
(Subscriber to /vision/decision)

```
self.esp32 = serial.Serial('/dev/ttyUSB0',
115200, timeout=1)

def send_command(self, command):
    self.esp32.write(command.encode('utf-8'))
    self.esp32.flush()
```

```
user@ros2-host:~$ ros2 node list
/ESP32_serial_node
/sensor_node
/vision_decision_node
/rviz2
/robot_state_publisher

def send_command(self, command):
self.esp32.write(command.encode('utf-8'))
self.esp32.flush()
```

```
user@ros2-host:~$ ros2 topic echo /vision/decision
---
data: 'L'
---
data: 'L'
---
data: 'C'
---
data: 'R'
---
data: 'R'
---
data: 'N'
---
data: 'N'
```

Prerequisite: Checking /dev/ttyUSB0 and dialout group

/dev/ttyUSB0

Embedded Environment

Receiving UTF-8 encoded characters



ESP32 DevKit

TRI

POLITEKNIK MALAYSIA
SULTAN SALAHUDDIN ABDUL AZIZ SHAH

Gemini Notebook

CLI Command Anatomy

Manual Execution
(4 Terminals)

```
user@ros2-host:~$
ros2 run mindspark_robot camera_node
```

```
user@ros2-host:~$
ros2 run mindspark_robot green_ball_node
```

```
user@ros2-host:~$
ros2 run mindspark_robot vision_decision_node
```

```
user@ros2-host:~$
ros2 run mindspark_robot esp32_control_node
```

Efficiency multiplier: Launch files consolidate node spin-ups, unify output streams, and eliminate multi-terminal dependency.

Orchestrated Execution
(1 Terminal)

```
user@ros2-host:~$
ros2 launch mindspark_robot pw3_hardware.launch.py
```

Launch File Orchestration

pw3_hardware.launch.py (Launch Description)

```python
def generate_launch_description():
    return LaunchDescription([
        Node(package='mindspark_robot', executable='camera_node'),
        Node(package='mindspark_robot', executable='green_ball_node'),
        Node(package='mindspark_robot', executable='vision_decision_node'),
        Node(package='mindspark_robot', executable='esp32_control_node'),
    ])
```

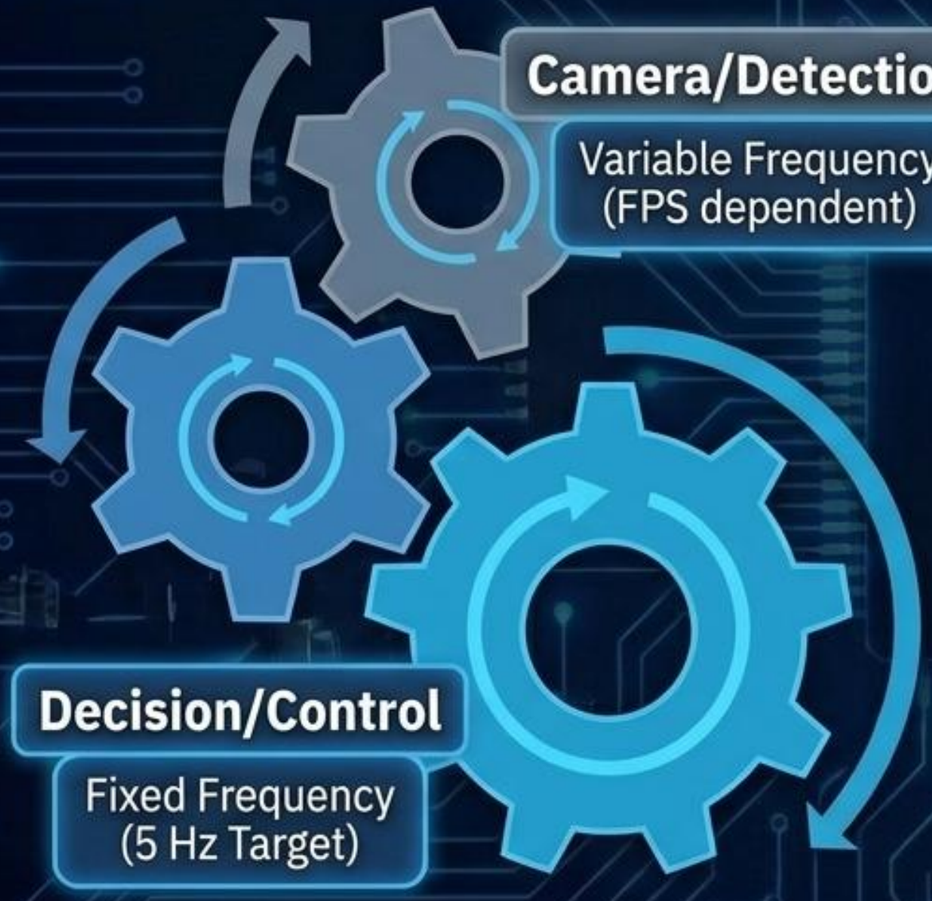
setup.py (Data Files Registration)

```python
data_files=[
    ...
    (os.path.join('share', package_name, 'launch'),
        glob('launch/*.launch.py')),
]
```

Auto-detects and registers all launch files.

Build requirement: Run `colcon build --symlink-install` from `~/mindspark_ws` after modifying launch or setup files.

Architectural Synthesis: Why Isolate Nodes?



Modularity

The ESP32 control logic is completely agnostic to how the detection works. You can swap the camera for a LiDAR node without touching the hardware code.

Frequency Bridging

Cameras fluctuate in FPS. The decision node normalizes output to a steady 5 Hz, preventing serial buffer flooding on the ESP32.

Fault Isolation

If the OpenCV detection crashes, the serial node remains active to safely turn off the ha

TRIC
TECHNOLOGY OF ROBOTICS AND IOT CENTRE

The Cascading Timeout Protocol

Event 1 (t=0s)

camera_node obstructed. /green_ball stops publishing valid coordinates.

Event 2 (t=1.0s)

vision_decision_node exceeds timeout. Automatically forces /vision/decision message to 'N'.

Event 3 (t=2.0s)

If /vision/decision topic crashes entirely, esp32_control_node exceeds timeout. Serial Node independently sends 'N' to the hardware buffer.

Hardware State:
ESP32 triggers allLEDsOff(). Actuators are safed.

Without independent watchdogs, an abrupt laptop failure could leave heavy robotics hardware locked in an ON state. Timeouts ensure default-to-safe behavior.

TRIC
TECHNOLOGY OF ROBOTICS AND IOT CENTRE